Лабораторная №7. Добавление поддержки SPI интерфейса

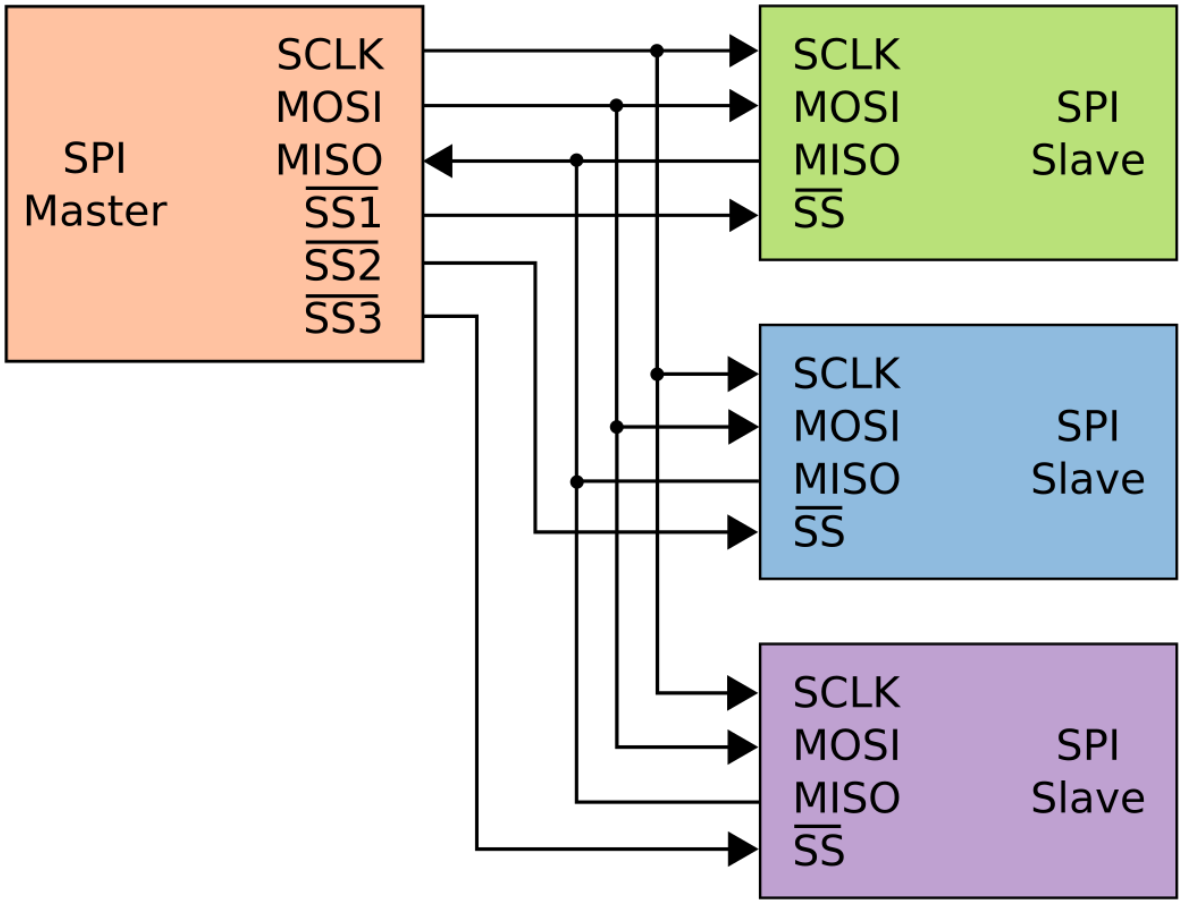
Цель работы

Изучить принципы работы SPI-интерфейса в одноплатных компьютерах под управлением Linux, освоить методы конфигурирования ядра и модификации Device Tree для добавления поддержки нового аппаратного интерфейса, а также получить практические навыки взаимодействия с SPI-устройствами из пользовательского пространства.

Подготовительный материал

Краткая теория о самом интерфейсе

SPI (Serial Peripheral Interface) — это интерфейс для связи с периферийными устройствами, такими как датчики, преобразователи и память.

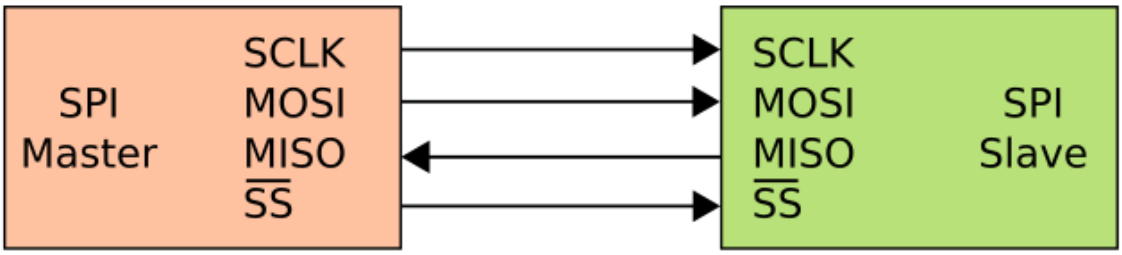


Архитектура интерфейса строится по принципу «ведущий-ведомый» (master-slave), где ведущий формирует тактовый сигнал синхронизации (SCK). Передача данных осуществляется по двум линиям: MOSI (Master Out Slave In) — данные от ведущего к ведомому, и MISO (Master In Slave

Out) — данные от ведомого к ведущему. Это позволяет реализовать полнодуплексный обмен информацией. Важной особенностью интерфейса является то, что при каждой записи команды или данных ведомое устройство всегда возвращает ответ той же длины, даже если этот ответ не содержит полезной информации.

Классическая схема подключения, приведенная на рисунке выше, является параллельной: все ведомые устройства совместно используют линии данных (MOSI, MISO) и тактирования (SCK). Единственным исключением является сигнал выбора ведомого (~CS), который для каждого устройства должен быть индивидуальным. На рисунке эти сигналы обозначены как SSx. Для их формирования можно использовать как специализированные выводы SPI-контроллера, так и универсальные GPIO-пины.

В случае с платой Lichee RV Dock базово предоставляется возможность подключить только одно SPI-устройство, что упрощает схему (см. рисунок ниже).



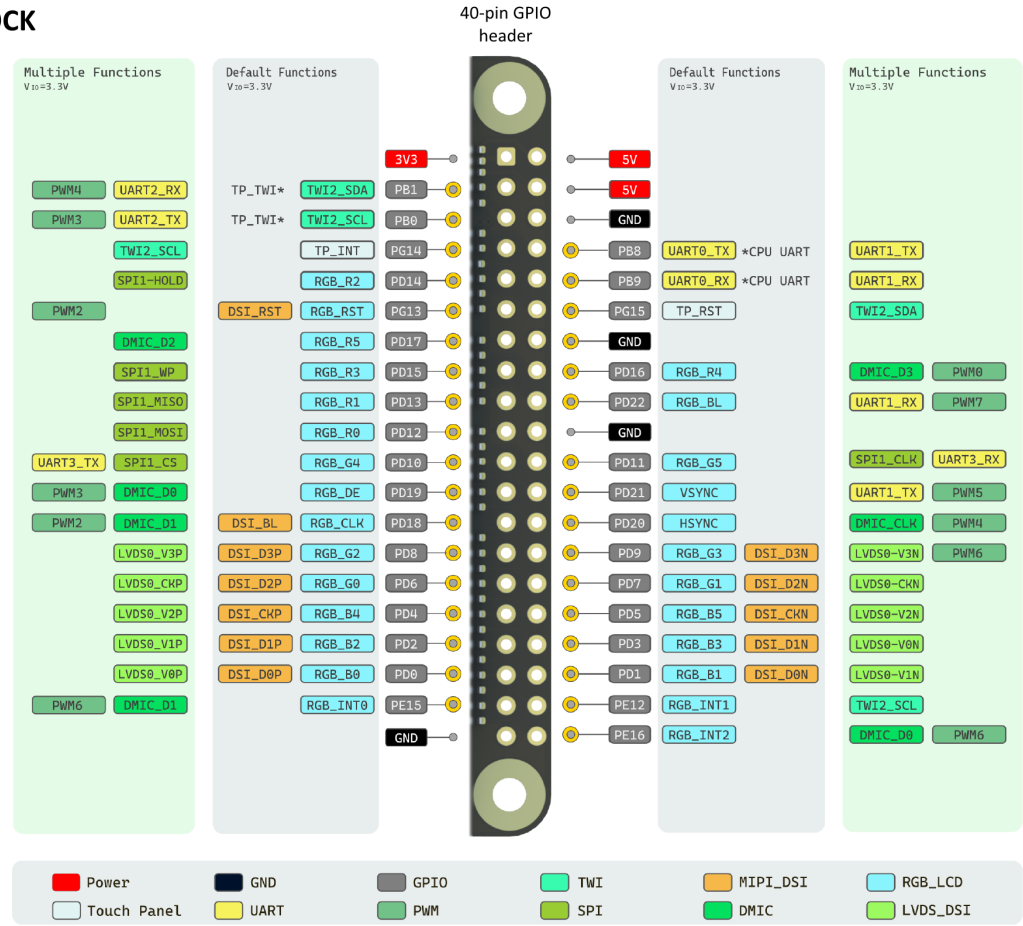
Интерфейс SPI применяется преимущественно во встроенных системах и электронных устройствах для короткой дистанции, обеспечивая высокоскоростной синхронный обмен данными между микроконтроллером (мастером) и периферийными микросхемами (ведомыми). Основные цели его использования — подключение и управление разнообразной периферией, такой как датчики (температуры, давления, акселерометры), карты памяти (SD, MMC), ЦАП/АЦП, сдвиговые регистры, драйверы дисплеев и тд.

Постановка задачи

Основной целью лабораторной работы является обеспечение возможности взаимодействия с SPI-интерфейсом микроконтроллера Allwinner D1 из пользовательского пространства операционной системы Linux.

Для достижения данной цели необходимо решить ряд задач, возникающих при анализе аппаратной конфигурации исследуемой платы. При обращении к распиновке (схема или пин-аут) платы Lichee RV Dock можно обнаружить следующую особенность:

Sipeed Lichee RV DOCK pinout diagram



SPI-интерфейс выведен на GPIO-разъем в качестве альтернативной функции. Это означает, что по умолчанию соответствующие выводы процессора сконфигурированы для работы с RGB-экраном. Следовательно, первая задача формулируется следующим образом: сконфигурировать выводы микроконтроллера для работы в режиме SPI-интерфейса.

Вторая задача связана с отсутствием в стандартной конфигурации ядра необходимых драйверов для поддержки SPI-интерфейса семейства микроконтроллеров sun20i (Allwinner D1). Таким образом, вторая задача может быть сформулирована так: добавить в ядро драйверы, обеспечивающие работу SPI-интерфейса в пользовательском пространстве.

Необходимые шаги

Для успешного решения поставленных задач необходимо выполнить следующие действия:

- Добавление и активация интерфейса в Device Tree
- Добавление драйвера поддержки интерфейса SPI в ядро
- Добавление сопутствующих драйверов

Добавление и активация интерфейса в Device Tree

Для понимания процесса формирования бинарных .dtb файлов из исходных текстов дерева устройств необходимо рассмотреть структуру соответствующих файлов для платы Lichee RV Dock.

В директории исходных кодов ядра arch/riscv/boot/dts/allwinner/ расположены исходные тексты дерева устройств для одноплатных компьютеров на базе чипа Allwinner D1.

При анализе файла *sun20i-d1-lichee-rv-dock.dts* можно заметить включение в него аналогичного файла *sun20i-d1-lichee-rv.dts* для базовой версии одноплатного компьютера. В свою очередь, данный файл содержит включение файлов с более общими параметрами. В результате последовательного включения достигается файл *sunxi-d1s-t113.dtsi*, в котором объявлены все интерфейсы и устройства, поддерживаемые микроконтроллером D1. Файлы с расширением .dtsi выполняют функцию заголовочных файлов для исходных файлов формата .dts, где суффикс *i* обозначает include.

В заголовочном файле содержатся следующие объявления, относящиеся к SPI-интерфейсу:

Объявление выводов микроконтроллера, поддерживающих работу двух SPI-интерфейсов (spi0 и spi1). Соответствие объявлений аппаратной конфигурации можно проверить по схеме одноплатного компьютера:

```
/omit-if-no-ref/
spi0_pins: spi0-pins {
        pins = "PC2", "PC3", "PC4", "PC5";
        function = "spi0";
};

/omit-if-no-ref/
spi1_pb_pins: spi1-pb-pins {
        pins = "PB0", "PB8", "PB9", "PB10", "PB11", "PB12";
        function = "spi1";
};

/omit-if-no-ref/
spi1_pd_pins: spi1-pd-pins {
        pins = "PD10", "PD11", "PD12", "PD13", "PD14", "PD15";
        function = "spi1";
};
```

Важно: директива /omit-if-no-ref/ предотвращает включение в результирующий .dtb файл объектов, которые не используются после объявления.

Объявление SPI интерфейсов,в которых указано с какими аппаратными прерываниями, интерфейсами и железом должен пользоваться интерфейс:

```
spi0: spi@4025000 {
        compatible = "allwinner,sun20i-d1-spi",
                        "allwinner,sun50i-r329-spi";
        reg = <0x4025000 0x1000>;
        interrupts = <31 IRQ_TYPE_LEVEL_HIGH>;
        clocks = <&ccu CLK_BUS_SPI0>, <&ccu CLK_SPI0>;
        clock-names = "ahb", "mod";
        resets = <&ccu RST_BUS_SPI0>;
```

```
        dmas = <&dma 22>, <&dma 22>;
        dma-names = "rx", "tx";
        num-cs = <1>;
        status = "disabled";
        #address-cells = <1>;
        #size-cells = <0>;
};

spi1: spi@4026000 {
        compatible = "allwinner,sun20i-d1-spi-dbi",
                        "allwinner,sun50i-r329-spi-dbi",
                        "allwinner,sun50i-r329-spi";
        reg = <0x4026000 0x1000>;
        interrupts = <32 IRQ_TYPE_LEVEL_HIGH>;
        clocks = <&ccu CLK_BUS_SPI1>, <&ccu CLK_SPI1>;
        clock-names = "ahb", "mod";
        resets = <&ccu RST_BUS_SPI1>;
        dmas = <&dma 23>, <&dma 23>;
        dma-names = "rx", "tx";
        num-cs = <1>;
        status = "disabled";
        #address-cells = <1>;
        #size-cells = <0>;
};
```

Параметр compatible содержит идентификаторы драйверов, поддерживающих работу с соответствующим устройством. Первым указывается предпочтительный драйвер для инициализации устройства. При загрузке ядра драйверы анализируют значение данного поля у активных устройств и при обнаружении поддерживаемого устройства вызывают функцию probe для его инициализации в операционной системе.

Обратите внимание, что статус обоих устройств имеет значение "disabled". Для активации интерфейса необходимо изменить данный статус в исходном файле платы, добавив следующую конфигурацию:

```
&spi1 {
        pinctrl-0 = <&spi1_pd_pins>;
        pinctrl-names = "default";
        status = "okay";
};
```

Данная запись активирует интерфейс и определяет используемые выводы для обмена данными.

Для обеспечения доступа к SPI-интерфейсу из пользовательского пространства необходим драйвер SPIDEV, поддержку которого требуется включить в конфигурацию ядра (параметр SPI_SPIDEV). Согласно документации, для автоматического подключения драйвера устройство должно присутствовать в одной из таблиц устройств, определенных в исходном коде драйвера. Существует два подхода к решению данной задачи:

- Добавление идентификатора устройства в одну из таблиц и компиляция модифицированного драйвера
- Использование существующего идентификатора из таблиц при объявлении устройства в Device Tree

В данной работе предлагается использовать второй подход:

```
&spi1{
        pinctrl-0 = <&spi1_pd_pins>;
        pinctrl-names = "default";
        status = "okay";
        spidev0: spidev@0 {
            compatible = "rohm,dh2228fv";
            spi-max-frequency = <100000000>;
            reg = <0x00>;
    };
```

На этом модификация Device Tree завершена.

Добавление драйвера поддержки интерфейса SPI в ядро

В качестве базовой конфигурации используется конфигурационный файл, полученный модификацией defconfig для Lichee RV Dock в лабораторной №4.

Базовая поддержка SPI-интерфейса (параметры SPI и SPI_MASTER) скорее всего уже присутствует в данной конфигурации. Однако для работы с конкретным SoC требуется соответствующий драйвер.

Несмотря на наличие в поле compatible заголовочного файла Device Tree соответствующих идентификаторов драйверов, поддержка семейства sun20i в стандартном драйвере отсутствовала. Решение данной проблемы было реализовано через патч, добавляющий поддержку SPI для ряда семейств процессоров Allwinner (описание изменений). Модификации затронули драйвер spi-sun6i.c, изначально предназначенный для устройств на архитектуре ARM. Ввиду схожей реализации SPI-интерфейса в драйвер была добавлена поддержка дополнительных семейств, включая sun20i.

Для включения поддержки необходимо активировать в конфигурации ядра параметр SPI_SUN6I. (**необходимо будет это сделать самостоятельно**)

Добавление сопутствующих драйверов

В процессе тестирования драйвера SPI_SUN6I было обнаружено, что инициализация устройства не происходит из-за невозможности доступа к DMA-каналам. Анализ показал необходимость наличия драйвера DMA для соответствующего семейства микроконтроллеров. Добавление драйвера DMA_SUN6I позволило успешно завершить инициализацию SPI-интерфейса. (**необходимо будет это сделать самостоятельно**)

Результат работы

После выполнения всех шагов в каталоге */dev/* появляется символьное устройство spidev. Для проверки работоспособности интерфейса можно выполнить тестирование путем замыкания выводов MOSI и MISO и запуска скрипта на языке Python:

```
import spidev
import time

def spi_sequential_test(bus=0, device=0, speed=1000000):
    try:
        spi = spidev.SpiDev()
        spi.open(bus, device)
        spi.max_speed_hz = speed
        spi.mode = 0
        spi.lsbfirst = False


        print("SPI последовательный тест запущен...")
        print("Отправляю данные: 0, 1, 2, 3, ...")
        print("-" * 40)

        for i in range(256):  # От 0 до 255
            # Отправляем одно число и получаем ответ
            data_to_send = [i]
            response = spi.xfer2(data_to_send)

            print(f"Отправлено: {data_to_send[0]:3d} (0x{data_to_send[0]:02X})
| "
                  f"Получено: {response[0]:3d} (0x{response[0]:02X}) | "
                  f"Бинарно: {response[0]:08b}")

            time.sleep(0.01)  # Небольшая пауза для анализатора

        spi.close()

    except Exception as e:

# Запуск теста
if __name__ == "__main__":
    # Настройте под вашу плату
    spi_sequential_test(bus=0, device=0, speed=1000000)
```

Результат:

```
[root@localhost my_spi_device]# python3 spi.py
SPI последовательный тест запущен...
Отправляю данные: 0, 1, 2, 3, ...
----------------------------------------
Отправлено:   0 (0x00) | Получено:   0 (0x00) | Бинарно: 00000000
Отправлено:   1 (0x01) | Получено:   1 (0x01) | Бинарно: 00000001
Отправлено:   2 (0x02) | Получено:   2 (0x02) | Бинарно: 00000010
Отправлено:   3 (0x03) | Получено:   3 (0x03) | Бинарно: 00000011
Отправлено:   4 (0x04) | Получено:   4 (0x04) | Бинарно: 00000100
Отправлено:   5 (0x05) | Получено:   5 (0x05) | Бинарно: 00000101
Отправлено:   6 (0x06) | Получено:   6 (0x06) | Бинарно: 00000110
Отправлено:   7 (0x07) | Получено:   7 (0x07) | Бинарно: 00000111
Отправлено:   8 (0x08) | Получено:   8 (0x08) | Бинарно: 00001000
Отправлено:   9 (0x09) | Получено:   9 (0x09) | Бинарно: 00001001
Отправлено:  10 (0x0A) | Получено:  10 (0x0A) | Бинарно: 00001010
```

Задание

Ознакомившись с подготовительным материалом к лабораторным №5, №6 и №7 решить следующие подзадачи:

- Проанализировать файлы Device Tree платы Lichee RV Dock и определить необходимые модификации для активации SPI-интерфейса
- Внести изменения в исходные файлы дерева устройств для активации SPI1 на выводах PD10-PD15
- Добавить в конфигурацию ядра поддержку:
  - SPI-интерфейса (убедиться в готовности параметров SPI и SPI_MASTER)
  - Драйвера SPI_SUN6I для работы с SoC Allwinner D1
  - Драйвера DMA_SUN6I для обеспечения доступа к DMA-каналам
  - Драйвера SPIDEV для работы с SPI из пользовательского пространства
- Собрать ядро с модифицированной конфигурацией и обновленными файлами дерева устройств
- Выполнить загрузку системы с новым ядром и проверить наличие устройства /dev/spidev*
- Провести тестирование работоспособности SPI-интерфейса методом обратной связи (замыкание MOSI и MISO)
- **Продемонстрировать работу преподавателю**
- Сформировать отчет о выполнении поставленных задач .doc и **выслать на почту преподавателя до обозначенного срока**